Robot Control Architectures

Nik Swoboda

nswoboda@fi.upm.es

Robótica y Percepción Computacional
Universidad Politécnica de Madrid

February 24, 2026

Robot Control Architectures?

We need to build robotic systems which can deal with the complexities of achieving goals while dealing with incomplete information, unexpected situations, ...

Robot control architectures provide a "design pattern" for managing these difficulties.

> *A robot control architecture provides guiding principles and constraints for organizing a robot's control system (its brain). It helps the designer to program the robot in a way that will produce the desired overall output behavior. (Matarić - The Robotics Primer)*

Robot Control Architectures?

Does programming a robot require the use of some control architecture?

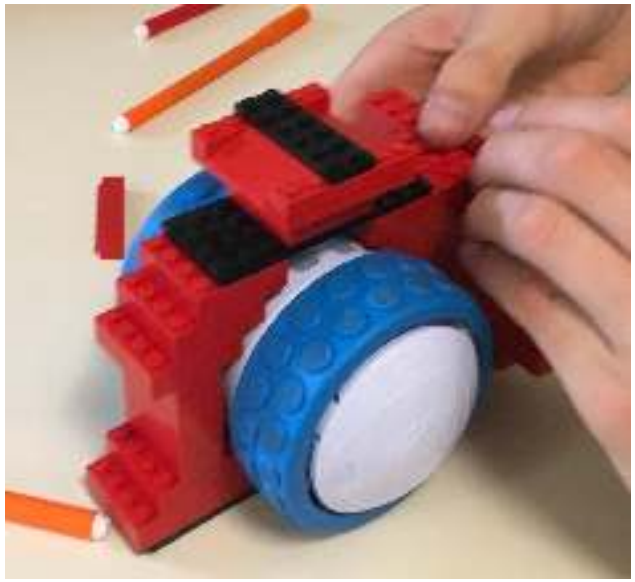
No, but as robots and the tasks that they perform become more complicated, having a control architecture becomes more useful.

Image from `https://www.nasa.gov/mission_pages/mars/multimedia/pia17273.html`.

The Basic Building Blocks

To make talking about these architectures easier lets "abstract" some of the basic functionality of a robot into "black boxes":

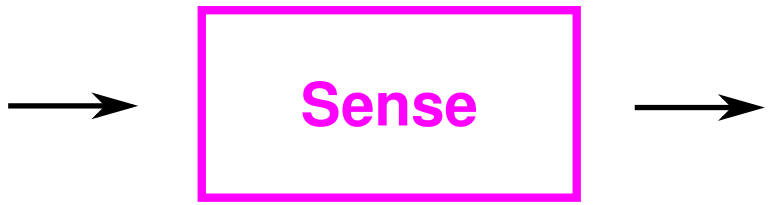
- Sense
- Plan
- Act



Have we forgotten anything?

Image from `https://commons.wikimedia.org/wiki/File:Construyendo_con_lego_y_robot_ollie.jpg`

The Basic Building Blocks - Sense



Module input: Raw sensor data

Module output: Environmental information

The Basic Building Blocks - Plan

Module input: Environmental and "known" information

Module output: A sequence of actions to perform

The Basic Building Blocks - Act

Module input: A sequence of actions to perform

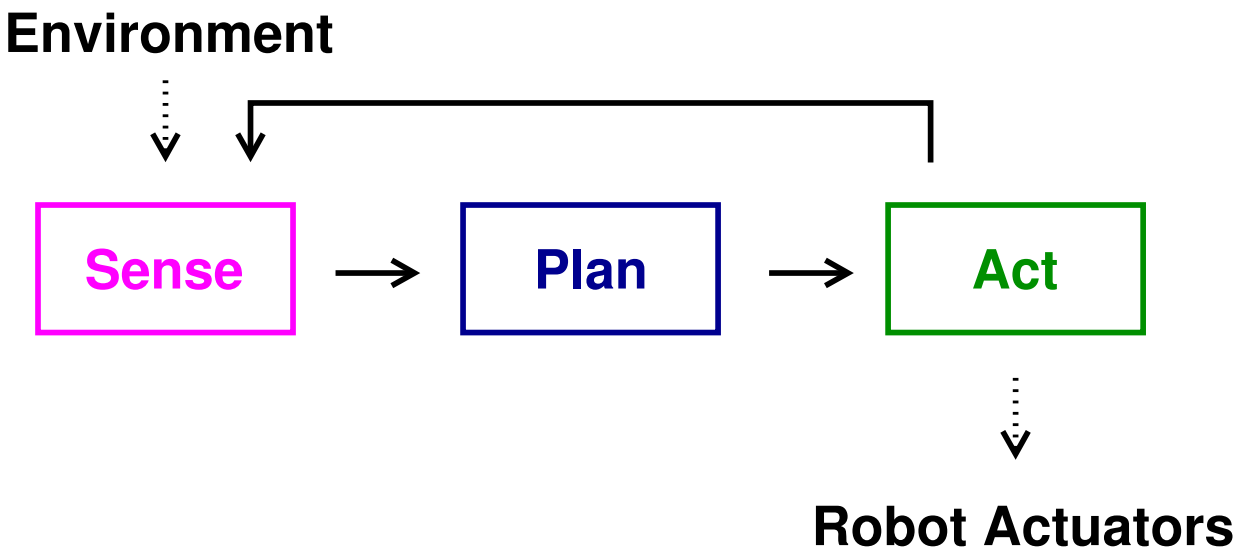
Module output: Actuator commands

Architectures - An Overview

- Deliberative control
- Reactive control
- Hybrid control

Note: this is not a definitive list, and sometimes other names are used.

Deliberative Control



Deliberative Control

Here "deliberative" = "thinking"

Sometimes called Hierarchical control.

Deliberative control systems are well suited for problems which require a lot of planning.

But in general they depend heavily on a sophisticated representation of the world.

Image from `https://pixy.org/3808791/`.

Challenges of Deliberative Control

This control paradigm has all problems associated with working with a *closed world assumption*.

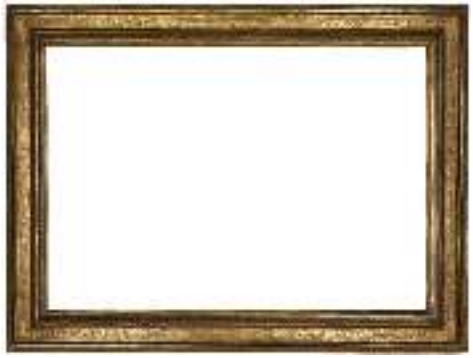
The model of the world needs to contain *everything* that the robot needs to know.

- A lot of pressure is put onto the programmer...
- The models could end up being *very* complicated and large.
- Model could be difficult to maintain.

Image from `https://pixabay.com/illustrations/earth-global-globe-school-3550553/`.

Challenges of Deliberative Control

This setting suffers from all of the difficulties related to the *frame problem*.



How do we manage change in the model of the world?

More specifically how do we know which states remain the same after some action. (Remember that we could have a very complex/large model.)

One simple solution is an assumption that everything remains the same unless explicitly described as the consequence of some action. BUT updating our world knowledge can still be a very complex process.

Image from `https://commons.wikimedia.org/wiki/File:Picture_frame_Wellcome_L0051764.jpg`.

Challenges of Deliberative Control

This approach uses what is known as *open loop control*. (Also known as 'ballistic control', and it is the opposite of closed loop or feedback control.)

Properties of the action are calculated once and then it is performed *without* the possibility of corrections.

We have no mechanism for *quick* response to a sudden unexpected change.

Image from `https://commons.wikimedia.org/wiki/File:De_Bange_155_mm_cannon_in_Moscow_(1).JPG`.

Deliberative Control

Disadvantages:

- Time to plan and space to model the search space
- Changes to the environment while planning
- Correctly executing a plan isn't a trivial matter

Where this control approach is used today:

- Applications where time doesn't matter, careful consideration is required, and the domain is static (e.g., certain kinds of robot surgery)
- Outside robotics, it is often used in game playing systems

Deliberative Control - An example

1966-1972 – Shakey the robot (Nils Nilssen, Charles Rosen et al. at SRI), the first robot to use "AI" (based upon STRIPS). (It started out with only 64K of memory, but later had 1.38MB)

Image from `http://en.wikipedia.org/wiki/File:Shakey.png`

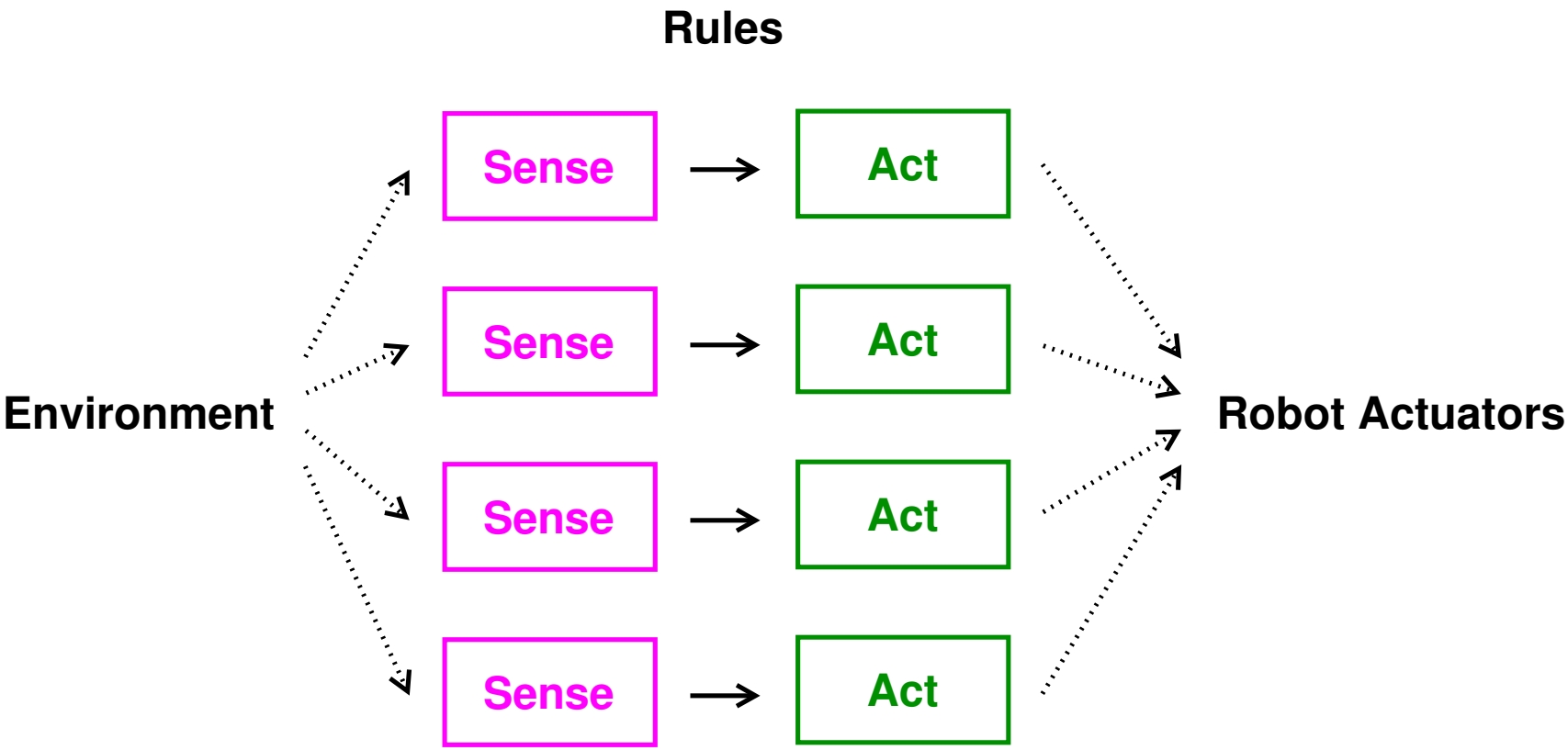
Deliberative Control - Some history

Purely deliberative systems were rather popular until around 1990.

Researchers who wanted to overcome the short comings of deliberative systems (usually with planning) began to look at biological systems (both animal and human) for inspiration in the late 70's.

In the late 1980's the reactive paradigm emerged from this work.

Reactive control



Reactive control

This architecture is characterized by "don't think, react".

In the most extreme cases these systems *do not* retain any long term model of the environment (nor do they contain explicitly represented goals).

In some sense the "hard thinking" is done by the designers not the robot. (Often times the hope is that the desired behaviors will emerge.)

Here one challenge can be the selection of the action to perform or how to combine actions from different rules. (Predicting the overall behavior can also be difficult.)

Image from `https://commons.wikimedia.org/wiki/File:Dachshund_catching_tennis_ball.jpg`.

Reactive control - Subsumption Architecture

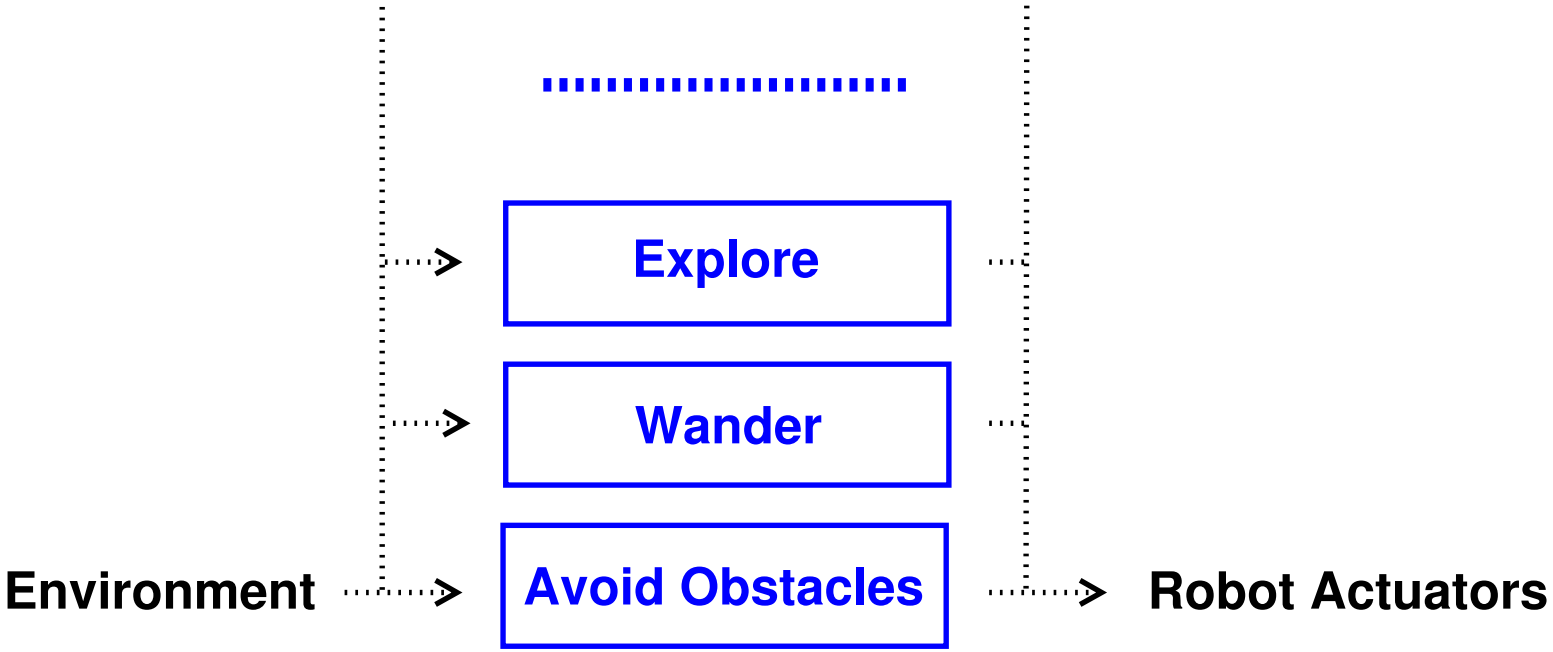
We can organize the rules by "intelligence", "competence" or "complexity". This is often called the *Subsumption Architecture* (Rodney Brooks).

Higher levels can *subsume* or assume the existence of lower levels.

Higher levels can also disable lower levels.

Generally don't have a notion of "internal state".

Reactive control - Subsumption Architecture



Reactive control - Subsumption Architecture

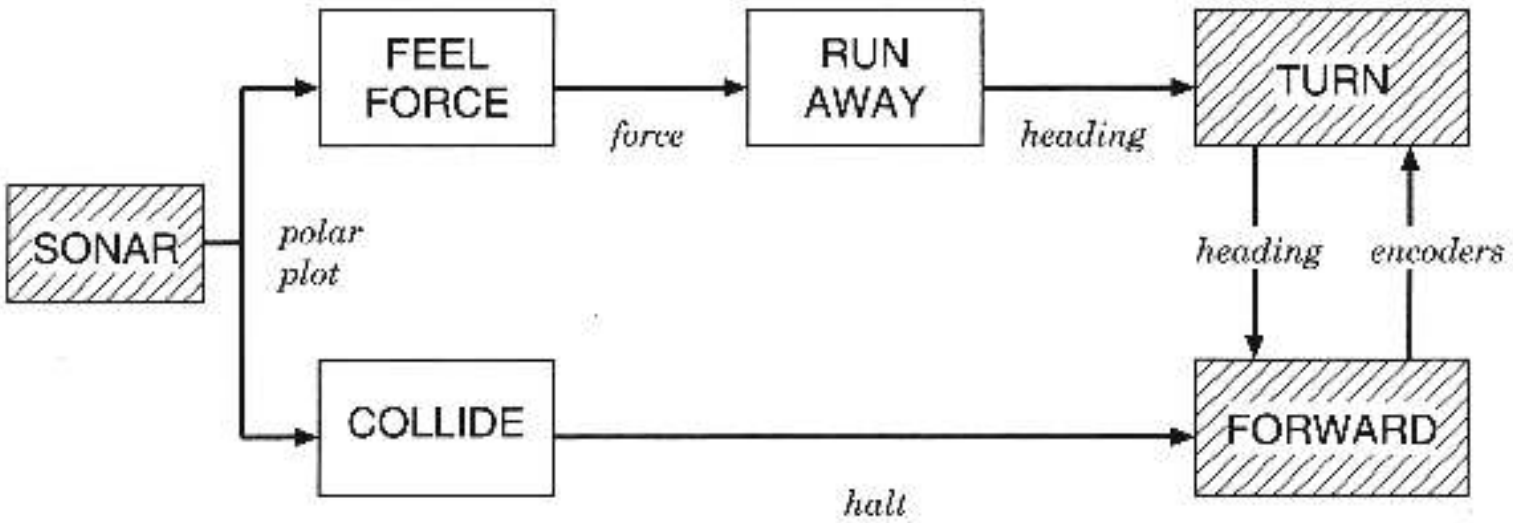


Figure 4.6 Level 0 in the subsumption architecture.

Image from *Introduction to AI Robotics* Robin Murphy, derived from Brooks' original paper

Reactive control - Subsumption Architecture

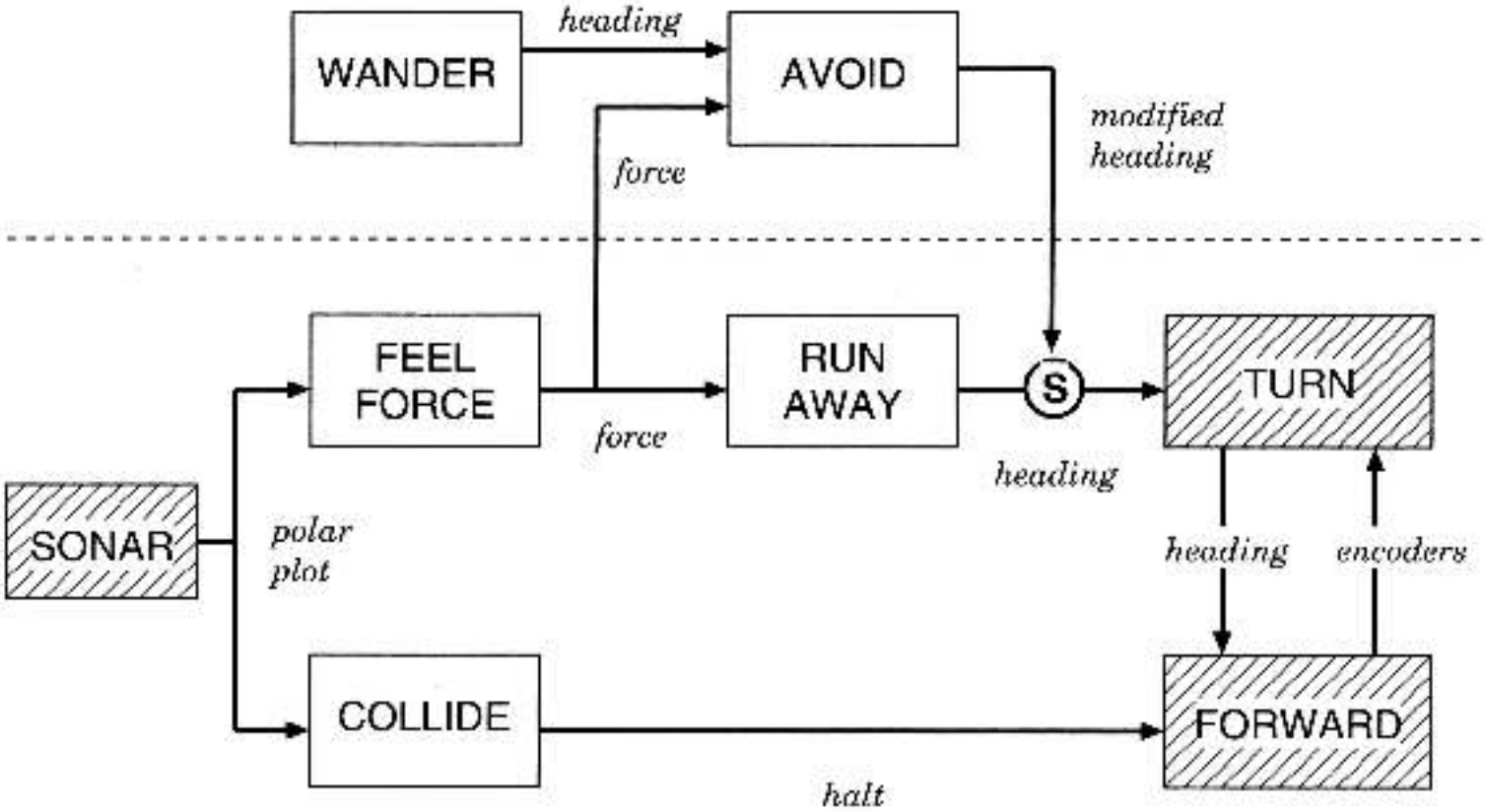


Figure 4.9 Level 1: wander.

Image from *Introduction to AI Robotics* Robin Murphy, derived from Brooks' original paper

Reactive control - Subsumption Architecture

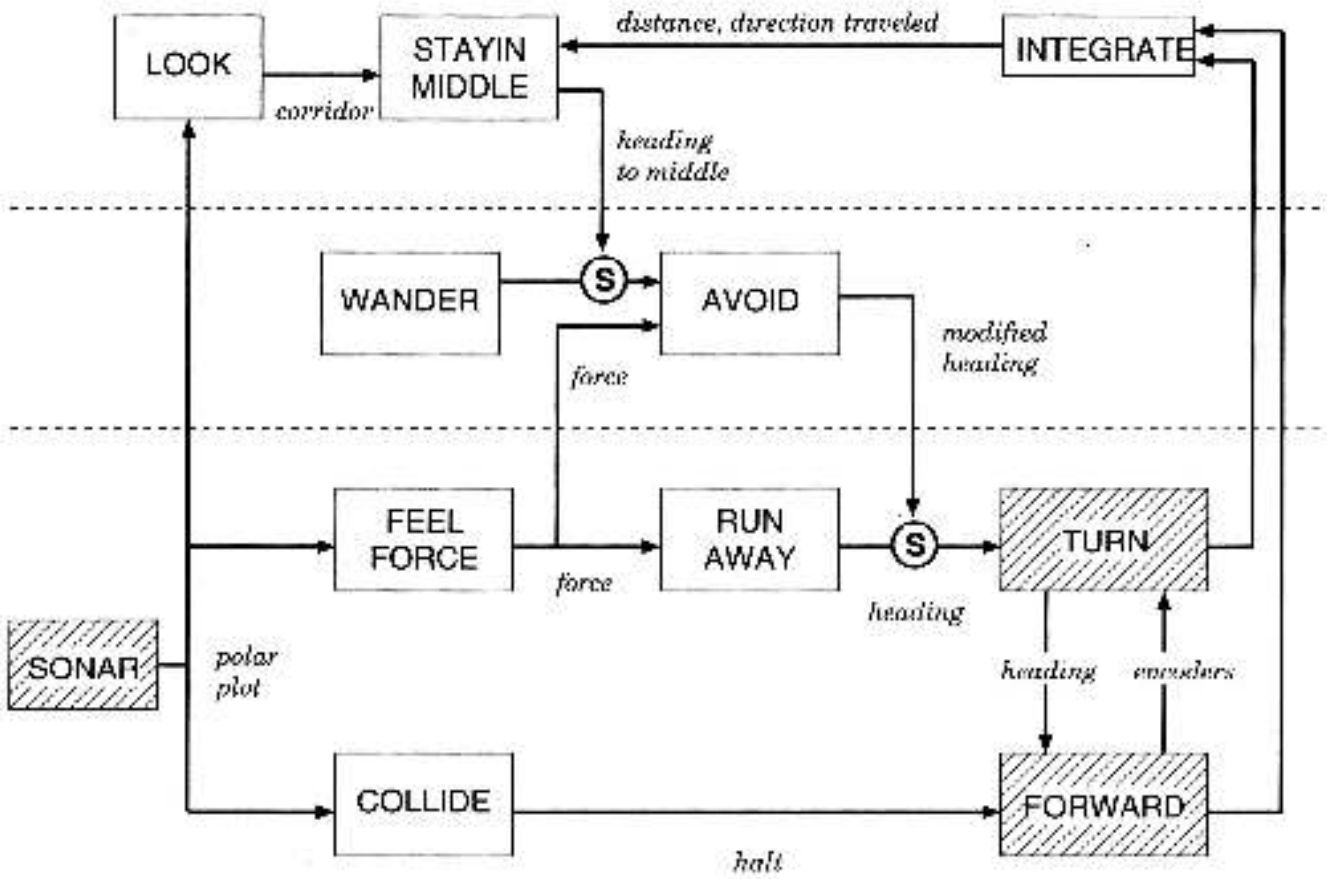


Figure 4.11 Level 2: follow corridors.

Image from *Introduction to AI Robotics* Robin Murphy, derived from Brooks' original paper

Reactive control

Advantages:

- Simple and modular (modules can be reused, support unit texting)
- Robust
- Can be embedded in hardware and run very quickly

Disadvantages:

- Inflexible, very task specific
- Can sometimes lack a vision of the “big picture”

Reactive control - An example

W. Grey Walter - Turtle (second generation, 1951)
(It was really designed before the term "Reactive Control" existed!)

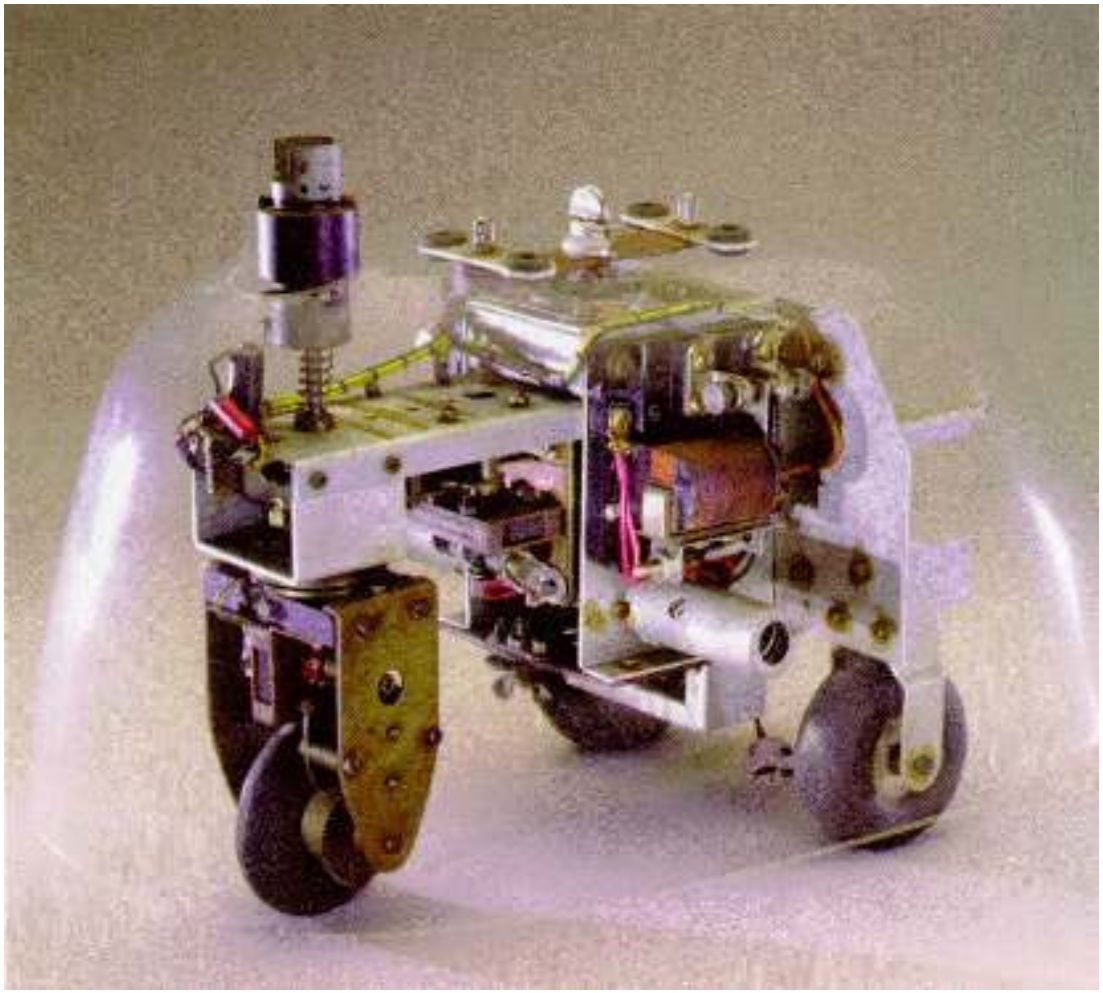


Image from `http://www.extremenxt.com/walter.htm`

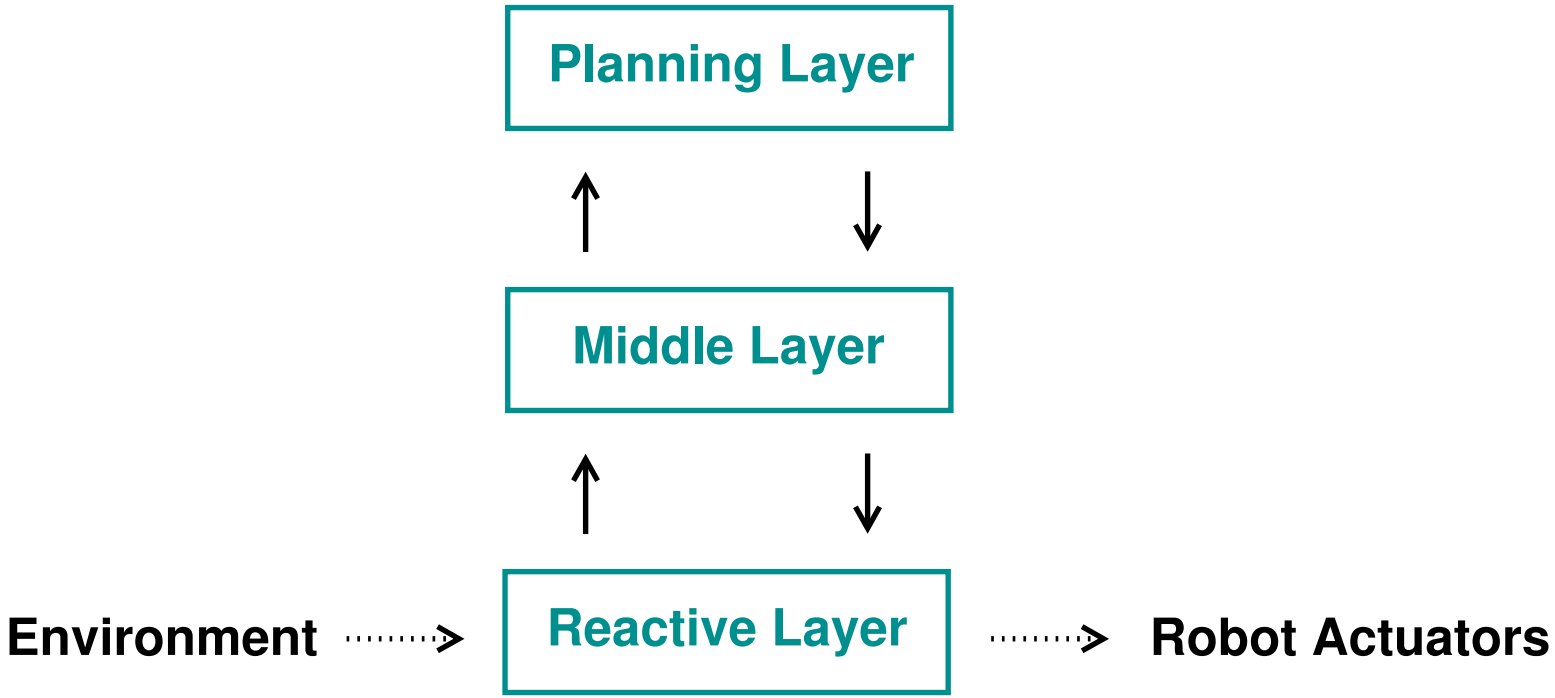
Hybrid control

Here the idea is to get the "best" of both worlds by combining deliberative and reactive control into the same system.

The reactive and deliberative layers are connected by a "middle layer" which coordinates their actions. (This can be difficult!)

Image from `https://www.sciencenews.org/article/hybrid-robot-merges-flier-two-snakelike-machines`.

Hybrid control



What Kind of Control Architecture?

The Roomba robotic vacuum cleaner cleaning a room (a 45 minute exposure).

Image from `http://commons.wikimedia.org/wiki/File:Roomba_time-lapse.jpg`

What Kind of Control Architecture?

In 1944 the first V-1 bomb was launched from the coast of Europe targeting London (in response to the Battle of Normandy starting with "D-Day"). It had a simple guidance system (weighted pendulum and gyrocompass) and an odometer (vane anemometer) to measure the distance traveled. (2,150 kg and 8.32m long)

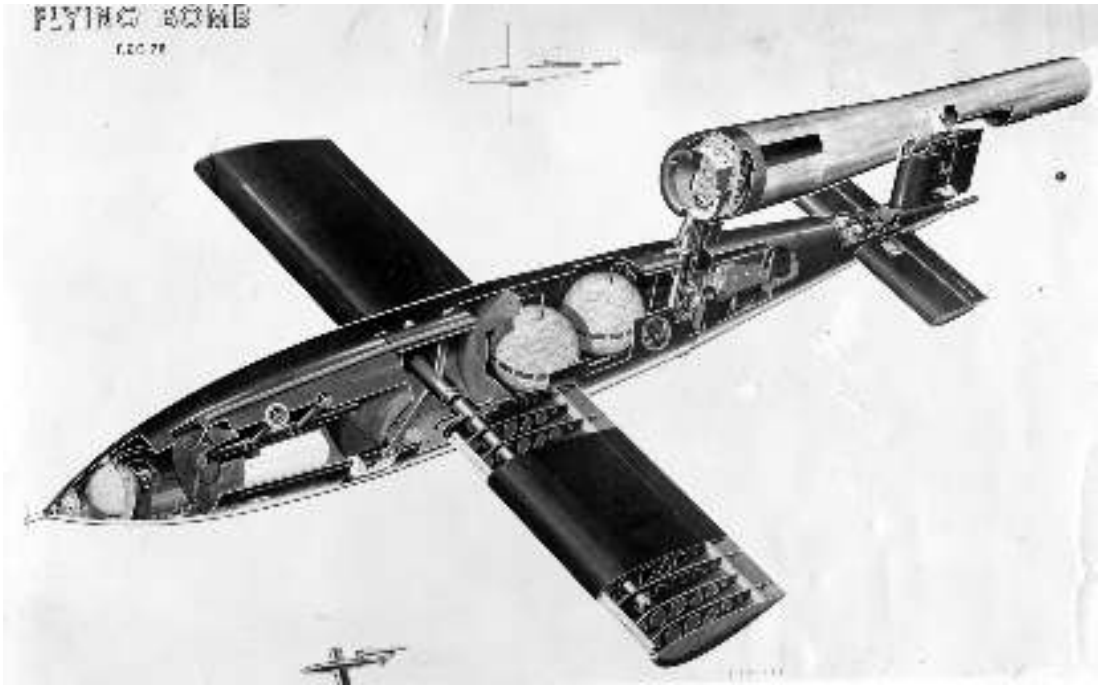


Image from `http://en.wikipedia.org/wiki/File:V-1_cutaway.jpg`

Putting it all together

What is the "best" robot control architecture?

In some sense that is like asking what is the "best" programming language to use to program a robot.

Then how do we know which architecture to use in a giving setting?

Image from `https://pixy.org/835632/`.